

นายอนันต์เอกก์ใหญ่พงศกร 673380430-9 sec.03

.\mvnw.cmd spring-boot:run

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6.1/jdom2-2.0.6.1.jar (328 kB at 1.0 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.10/jdependency-2.10.jar (416 kB at 1.2 MB/s)
[INFO] Attaching agents: []

:: Spring Boot ::
(v4.1.0)

2026-07-24T14:17:27.932+07:00 INFO 52476 --- [coffeemenu] [main] c.e.coffeemenu.CoffeemenuApplication : Starting CoffeemenuApplication using Java 24.0.1 with PID 52476 (C:\673380430-9\Lab05\coffeemenu\coffeemenu\target\classes started by computer in C:\673380430-9\Lab05\coffeemenu\coffeemenu)
2026-07-24T14:17:27.934+07:00 INFO 52476 --- [coffeemenu] [main] c.e.coffeemenu.CoffeemenuApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-24T14:17:28.336+07:00 INFO 52476 --- [coffeemenu] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-24T14:17:28.337+07:00 INFO 52476 --- [coffeemenu] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-24T14:17:28.338+07:00 INFO 52476 --- [coffeemenu] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-24T14:17:28.362+07:00 INFO 52476 --- [coffeemenu] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 394 ms
2026-07-24T14:17:28.533+07:00 INFO 52476 --- [coffeemenu] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-07-24T14:17:28.536+07:00 INFO 52476 --- [coffeemenu] [main] c.e.coffeemenu.CoffeemenuApplication : Started CoffeemenuApplication in 0.845 seconds (process running for 1.031s)
2026-07-24T14:20:06.417+07:00 INFO 52476 --- [coffeemenu] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-24T14:20:06.417+07:00 INFO 52476 --- [coffeemenu] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-24T14:20:06.418+07:00 INFO 52476 --- [coffeemenu] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

curl http://localhost:8080/coffees

Expected Result:

```
[{"id":1,"name":"Espresso","price":45.0}, {"id":2,"name":"Latte","price":55.0}]
```

```
PS C:\673380430-9\Lab05\coffeemenu> curl http://localhost:8080/coffees
```

```
Status: 200
StatusDescription:
Content: [{"id":1,"name":"Espresso","price":45.0}, {"id":2,"name":"Latte","price":55.0}]
RawContent: HTTP/1.1 200
Keep-Alive: timeout=60
Connection: keep-alive
Content-Length: 78
Content-Type: application/json
Date: Fri, 24 Jul 2026 07:20:06 GMT

[{"id":1,"name":"Espresso","price":45.0}, {"id":...
Forms: {}
Headers: [{"Keep-Alive", "timeout=60"}, [{"Connection", "keep-alive"}, [{"Content-Length", 78}, [{"Content-Type", "application/json"}]...}
Images: {}
InputFields: {}
Links: {}
ParsedHtml: mshtml.HTMLDocumentClass
RawContentLength: 78
```

curl http://localhost:8080/coffees/1 **Expected Result:** {"id":1,"name":"Espresso","price":45.0}

```
PS C:\673380430-9\Lab05\coffeemenu> curl http://localhost:8080/coffees/1
```

```
Status: 200
StatusDescription:
Content: {"id":1,"name":"Espresso","price":45.0}
RawContent: HTTP/1.1 200
Transfer-Encoding: chunked
Keep-Alive: timeout=60
Connection: keep-alive
Content-Type: application/json
Date: Fri, 24 Jul 2026 07:24:25 GMT

{"id":1,"name":"Espresso","price":45.0...
Forms: {}
Headers: [{"Transfer-Encoding", "chunked"}, [{"Keep-Alive", "timeout=60"}, [{"Connection", "keep-alive"}, [{"Content-Type", "application/json"}]...}
Images: {}
InputFields: {}
Links: {}
ParsedHtml: mshtml.HTMLDocumentClass
RawContentLength: 39
```

Invoke-RestMethod -Method Post -Uri http://localhost:8080/coffees -ContentType "application/json" -Body '{"name":"Cappuccino","price":60.0}'

Expected Result: Status 201 Created with {"id":3,"name":"Cappuccino","price":60.0}

```
PS C:\673380430-9\Lab05\coffeemenu> curl.exe -X POST http://localhost:8080/coffees -H "Content-Type: application/json" -d '{"name":"Cappuccino","price":60.0}'
{"timestamp":"2026-07-24T07:29:04.850Z","status":400,"error":"Bad Request","path":"/coffees"}
PS C:\673380430-9\Lab05\coffeemenu> Invoke-RestMethod -Method Post -Uri http://localhost:8080/coffees -ContentType "application/json" -Body '{"name":"Cappuccino","price":60.0}'

id name      price
-- ----
  3 Cappuccino 60.0
```

Invoke-RestMethod -Method Put -Uri http://localhost:8080/coffees/2 -ContentType "application/json" -Body '{"name":"Latte","price":50.0}'

Expected Output: Table showing updated Latte with price: 50.0

```
PS C:\673380430-9\Lab05\coffeemenu> Invoke-RestMethod -Method Put -Uri http://localhost:8080/coffees/2 -ContentType "application/json" -Body '{"name":"Latte","price":50.0}'

id name  price
-- ----
  2 Latte 50.0
```

Invoke-RestMethod -Method Delete -Uri <http://localhost:8080/coffees/3>

```
PS C:\673380430-9\Lab05\coffeemenu> Invoke-RestMethod -Method Delete -Uri http://localhost:8080/coffees/3

PS C:\673380430-9\Lab05\coffeemenu> Invoke-RestMethod -Uri http://localhost:8080/coffees

id name  price
-- ----
  1 Espresso 45.0
  2 Latte    50.0
```

Verification Summary

Endpoint	HTTP Method	Action / Result	Status
/coffees	GET	Retrieved initial coffee list	Passed
/coffees/1	GET	Retrieved single coffee by ID (1 Espresso)	Passed
/coffees	POST	Created new coffee (3 Cappuccino, 60.0)	Passed
/coffees/2	PUT	Updated Latte price from 55.0 to 50.0	Passed
/coffees/3	DELETE	Removed Cappuccino (id: 3)	Passed

Discussion

1. HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร

ยกตัวอย่างจากโปรเจกต์ตัวเอง

- GET: ใช้ดึง/อ่านข้อมูลจาก Server เช่น GET /coffees เพื่อดึงรายการกาแฟทั้งหมด
- POST: ใช้สร้าง/เพิ่มข้อมูลใหม่ลงใน Server เช่น POST /coffees เพื่อเพิ่มเมนู Cappuccino
- PUT: ใช้แก้ไข/อัปเดตข้อมูลเดิมที่มีอยู่แล้ว เช่น PUT /coffees/2 เพื่อแก้ไขราคาของ Latte
- DELETE: ใช้ลบข้อมูลออกจาก Server เช่น DELETE /coffees/3 เพื่อลบเมนูกาแฟ ID 3

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

- เหตุผลที่ต้องแยก: เพื่อทำตามหลัก *Separation of Concerns* โดยให้ Controller ทำหน้าที่จัดการเกี่ยวกับ HTTP Request/Response เท่านั้น ส่วน Service ทำหน้าที่ประมวลผล Business Logic และจัดการข้อมูล
- ข้อดีเมื่อโปรแกรมโตขึ้น: ช่วยให้ Code เป็นระเบียบ อ่านและดูแลรักษาง่าย (Maintainability) สามารถนำ Logic ใน Service ไปใช้ซ้ำได้ง่าย และสามารถเขียน Unit Test เพื่อทดสอบระบบได้สะดวกยิ่งขึ้น

3. ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากไม่ให้หายควรทำอย่างไร

- เวลาที่ข้อมูลหาย: ข้อมูลจะหายไปเมื่อทำการ Stop หรือ Restart ตัวแอปพลิเคชัน/เซิร์ฟเวอร์ เนื่องจากข้อมูลถูกจัดเก็บไว้ในหน่วยความจำชั่วคราว (RAM)
- แนวทางแก้ไข: เปลี่ยนจากการเก็บใน List มาเชื่อมต่อกับระบบจัดเก็บข้อมูลแบบถาวร (Persistent Storage) เช่น ฐานข้อมูล (Database) อย่าง H2, MySQL, PostgreSQL หรือใช้ Spring Data JPA ในการบันทึกข้อมูลลงดิสก์

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

- **@RestController**: ระบุว่า Class นี้คือ Web Controller สำหรับสร้าง RESTful API โดยจะแปลง Return Value ส่งกลับเป็น JSON/XML โดยอัตโนมัติ
- **@GetMapping** / **@PostMapping**: ใช้จับคู่ (Map) HTTP Request แบบ GET และ POST เข้ากับ Method ที่กำหนด
- **@PathVariable**: ใช้ดึงค่าพารามิเตอร์ที่ส่งมาทาง URL Path เช่น ตัวเลข id จาก /coffees/{id}
- **@RequestBody**: ใช้แปลงข้อมูล JSON ที่ส่งมาใน HTTP Request Body ให้กลายเป็น Java Object